msfasha /
307304-Data-Mining

Code Issues Pull requests Actions Projects Wiki Security In

main

307304-Data-Mining / Part 5-Associatin Rules Mining / 5-1-association_rules_mining.ipynb

msfasha modify associatino rules mining bed9e1c · now

801 lines (801 loc) · 31.9 KB

Open in Colab YouTube

1. Introduction to Association Rule Mining - Market Baset Analysis (MBA)

- Association rule learning is a rule-based (if-then-else) machine learning method for discovering interesting relations between variables in large databases.
- It is intended to identify strong rules discovered in databases using some measures of interestingness
- In any given transaction with a variety of items, association rules are meant to discover the rules that determine how or why certain items are connected.
- Market Basket Analysis (MBA) is a specific case of Association Rule Mining, hence, it is a data mining technique used to discover patterns and relationships between items purchased together in transactions. - - It helps identify combinations of products that are frequently bought together, providing valuable insights for businesses to optimize sales, marketing, and inventory strategies.
- Based on the concept of strong rules, Rakesh Agrawal, Tomasz Imieliński and Arun Swami[2] introduced association rules for discovering regularities between products in large-scale transaction data recorded by point-of-sale (POS) systems in supermarkets. For example, the rule:

$$\{\text{onions}, \text{potatoes}\} \Rightarrow \{\text{burger}\}$$

found in the sales data of a supermarket would indicate that if a customer buys onions and potatoes together, they are likely to also buy hamburger meat. Such information can be used as the basis for decisions about marketing activities such as, e.g., promotional pricing or product placements.

- In addition to the above example from market basket analysis, association rules are employed today in many application areas including Web usage mining, intrusion detection, continuous production, and bioinformatics. In contrast with sequence mining, association rule learning typically does not consider the order of items either within a transaction or across transactions.

- The association rule algorithm itself consists of various parameters that can make it difficult for those without some expertise in data mining to execute, with many rules that are arduous to understand.

reference: https://en.wikipedia.org/wiki/Association_rule_learning

Key Concepts:

1. **Association Rules**:
   - These are if-then rules that identify relationships between items. For example, "If a customer buys bread, they are likely to buy butter."

2. **Metrics**:
   - **Support**: Measures how often an item set appears in the dataset.

$$\text{Support(A)} = \frac{\text{Number of transactions containing A}}{\text{Total transactions}}$$

Valid Values for Support

- **Range**:
  The value of support ranges from **0 to 1**:
  - **0**: The itemset does not appear in any transaction.
  - **1**: The itemset appears in all transactions.
- **Threshold**:
  In practice, a minimum support threshold is specified to filter out infrequent itemsets, ensuring only itemsets that appear frequently in the dataset are considered for further analysis.

Types of Support

1. **Absolute Support**:
   - Represents the actual count of transactions that contain the itemset.
   - Example: If $X = \{bread, milk\}$ appears in 50 transactions, the absolute support is 50.
2. **Relative Support**:
   - Expresses support as a proportion of the total number of transactions.
   - Example: If X appears in 50 transactions out of 500, the relative support is $\frac{50}{500} = 0.1$.

Interpreting Support

- **High support**: The itemset is common and may represent a strong association.
- **Low support**: The itemset is rare and might not provide actionable insights.

Choosing Support Thresholds

- Setting a very high support threshold can exclude rare but potentially interesting itemsets.
- A very low support threshold might lead to too many itemsets, making the analysis computationally expensive and cluttered.

- **Confidence**: Measures how often the rule is true.

$$\text{Confidence}(\text{A} \rightarrow \text{B}) = \frac{\text{Support}(\text{A} \cap \text{B})}{\text{Support}(\text{A})}$$

Valid Values for Confidence

- **Range**: Confidence values range from **0 to 1**:
  - **0**: The rule $X \rightarrow Y$ never holds true.
  - **1**: The rule $X \rightarrow Y$ always holds true (perfect confidence).

Interpreting Confidence

1. **High Confidence (≈ 1)**:
   - Indicates that Y almost always occurs when X occurs.
   - Example: If $X = \{\text{bread}\}$ and $Y = \{\text{butter}\}$, a high confidence implies that customers buying bread are very likely to buy butter.
2. **Low Confidence (≈ 0)**:
   - Indicates that Y rarely occurs when X occurs.
   - Example: If $X = \{\text{bread}\}$ and $Y = \{\text{cereal}\}$, a low confidence implies that buying bread is not strongly associated with buying cereal.

Confidence vs. Other Metrics

- **Confidence** does not consider how common Y is overall. This can lead to misleading rules if Y is very frequent in the dataset. For example, a high-confidence rule might occur simply because Y is common, not because X and Y are strongly related.
- To address this, metrics like **lift** or **conviction** are often used alongside confidence.

Thresholds in Practice

- A **minimum confidence threshold** is typically set to filter rules that meet a desired level of predictive strength.
- A common approach is to start with rules that have both high confidence (e.g., $\text{Confidence} > 0.7$) and high support.

---

main 307304-Data-Mining / Part 5-Associatin Rules Mining / 5-1-association_rules_mining.ipynb ↑ Top

Preview Code Blame Raw

In the Apriori algorithm, **lift** is a metric used to evaluate the strength of a rule in association rule mining. It is calculated as:

$$\text{Lift}(X \to Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X) \times \text{Support}(Y)}$$

Valid Values for Lift

1. **Greater than 1**:
   - Indicates a **positive association** between X and Y.
   - X and Y are more likely to occur together than if they were independent.
2. **Equal to 1**:
   - Indicates **no association** between X and Y.
   - X and Y occur together just as often as would be expected under independence.
3. **Less than 1**:
   - Indicates a **negative association** between X and Y.
   - X and Y are less likely to occur together than if they were independent.

When using the Apriori algorithm, a lift threshold is often set to filter rules that meet a certain level of interest, typically focusing on those with $\text{Lift} > 1$.

---

3. **Apriori Algorithm**:

   This is the most common algorithm used for MBA. It identifies frequent itemsets and builds association rules from them by leveraging the principle that subsets of frequent itemsets must also be frequent.

Applications:

- **Retail**: Arranging products in stores to increase cross-selling (e.g., placing chips near sodas).
- **E-commerce**: Personalizing recommendations (e.g., Amazon's "Customers who bought this also bought").
- **Inventory Management**: Predicting product demand and reducing stockouts.

2. **Step-by-Step Execution of the Apriori Algorithm**

We will go step-by-step through how the **Apriori algorithm** would process this dataset. We'll compute frequent itemsets and generate rules using the given data, along with thresholds for **support**, **confidence**, and **lift**.

Let's assume a small dataset with 12 transactions:

Transaction ID	Item 1	Item 2	Item 3	Item 4	Item 5
1	Milk	Egg	Bread	Butter	
2	Milk	Butter	Egg	Ketchup	
3	Bread	Butter	Ketchup		
4	Milk	Bread	Butter		
5	Bread	Butter	Cookies		
6	Milk	Bread	Butter	Cookies	
7	Milk	Cookies			
8	Milk	Bread	Butter		
9	Bread	Butter	Egg	Cookies	
10	Milk	Butter	Bread		
11	Milk	Bread	Butter		
12	Milk	Bread	Cookies	Ketchup	

We set out thresholds as following:

- **Minimum Support**: 50% (focus on items present in at least half of the transactions).
- **Minimum Confidence**: 70%.
- **Lift**: Lift > 1 for actionable rules.

Step 1: Dataset Transformation

Convert the dataset into a binary format:

Transaction	Milk	Egg	Bread	Butter	Ketchup	Cookies
1	1	1	1	1	0	0
2	1	1	0	1	1	0
3	0	0	1	1	1	0
4	1	0	1	1	0	0
5	0	0	1	1	0	1
6	1	0	1	1	0	1
7	1	0	0	0	0	1
8	1	0	1	1	0	0
9	0	1	1	1	0	1
10	1	0	1	1	0	0
11	1	0	1	1	0	0

11	1	0	1	1	0	0
12	1	0	1	0	1	1

Step 2: Identify Frequent 1-Itemsets

The algorithm starts by calculating the support for each single item.

Item	Support Count	Support (%)	Frequent?
Milk	9	75%	Yes
Egg	3	25%	No
Bread	10	83%	Yes
Butter	10	83%	Yes
Ketchup	3	25%	No
Cookies	5	42%	No

Frequent 1-itemsets: `{Milk}`, `{Bread}`, `{Butter}`.

Step 3: Generate 2-Itemsets

Now, combine frequent 1-itemsets into 2-itemsets and calculate their support.

Itemset	Support Count	Support (%)	Frequent?
{Milk, Bread}	7	58%	Yes
{Milk, Butter}	7	58%	Yes
{Bread, Butter}	9	75%	Yes

Frequent 2-itemsets: `{Milk, Bread}`, `{Milk, Butter}`, `{Bread, Butter}`.

Step 4: Generate 3-Itemsets

Combine frequent 2-itemsets into 3-itemsets and calculate their support.

Itemset	Support Count	Support (%)	Frequent?
{Milk, Bread, Butter}	6	50%	Yes

Frequent 3-itemset: `{Milk, Bread, Butter}`.

Step 5: No Larger Itemsets

Since there are no frequent 4-itemsets (support would drop below 50%), we stop here.

Step 6: Generate Association Rules

Now, use the frequent itemsets to generate association rules and calculate their metrics:

Now, use the frequent itemsets to generate association rules and calculate their metrics: **support**, **confidence**, and **lift**.

Rules from `{Milk, Bread, Butter}`:

1. **Rule**:

$$\text{Milk, Bread} \rightarrow \text{Butter}$$

- **Support**:

$$P(\text{Milk, Bread, Butter}) = 6/12 = 50\%$$

- **Confidence**:

$$P(\text{Butter}|\text{Milk, Bread}) = \frac{P(\text{Milk, Bread, Butter})}{P(\text{Milk, Bread})} = \frac{6}{7} = 85\%$$

- **Lift**:

$$\text{Lift} = \frac{P(\text{Butter}|\text{Milk, Bread})}{P(\text{Butter})} = \frac{0.85}{0.83} \approx 1.02$$

2. **Rule**:

$$\text{Milk, Butter} \rightarrow \text{Bread}$$

- **Support**:

$$P(\text{Milk, Bread, Butter}) = 6/12 = 50\%$$

- **Confidence**:

$$P(\text{Bread}|\text{Milk, Butter}) = \frac{P(\text{Milk, Bread, Butter})}{P(\text{Milk, Butter})} = \frac{6}{7} = 85\%$$

- **Lift**:

$$\text{Lift} = \frac{P(\text{Bread}|\text{Milk, Butter})}{P(\text{Bread})} = \frac{0.85}{0.83} \approx 1.02$$

3. **Rule**:

$$\text{Bread, Butter} \rightarrow \text{Milk}$$

- **Support**:

$$P(\text{Milk, Bread, Butter}) = 6/12 = 50\%$$

- **Confidence**:

$$P(\text{Milk}|\text{Bread, Butter}) = \frac{P(\text{Milk, Bread, Butter})}{P(\text{Bread, Butter})} = \frac{6}{9} \approx 66\%$$

- **Lift**:

$$\text{Lift} = \frac{P(\text{Milk}|\text{Bread, Butter})}{P(\text{Milk})} = \frac{0.66}{0.75} \approx .88$$

Final Results

Frequent Itemsets:

- **1-itemsets**: `{Milk}`, `{Bread}`, `{Butter}`
- **2-itemsets**: `{Milk, Bread}`, `{Milk, Butter}`, `{Bread, Butter}`
- **3-itemset**: `{Milk, Bread, Butter}`

Rules:

Rule	Support	Confidence	Lift	Actionable?
$\text{Milk, Bread} \rightarrow \text{Butter}$	50%	85%	1.02	Yes
$\text{Milk, Butter} \rightarrow \text{Bread}$	50%	85%	1.02	Yes
$\text{Bread, Butter} \rightarrow \text{Milk}$	50%	66%	.88	No

Interpretation and Actionable Insights

Rule	Explaniation
$\text{Milk, Bread} \rightarrow \text{Butter}$	Customers buying Milk and Bread are highly likely (85%) to also buy Butter. Consider bundling these items.
$\text{Milk, Butter} \rightarrow \text{Bread}$	Strong association; placing these items together could increase sales.
$\text{Bread, Butter} \rightarrow \text{Milk}$	Suggests that Milk is a complementary product to Bread and Butter, Weak lift thought

---

3. **Python Code Example**

We will implement the **Apriori algorithm** using the `mlxtend` library for frequent itemset mining and rule generation.

a. Install Required Libraries

Make sure you have the required libraries installed:

```
pip install pandas mlxtend
```

b. Load the Dataset

We will represent the dataset as a binary **transaction matrix** where each row is a transaction, and each column is an item (1 = purchased, 0 = not purchased).

In [47]:

```
import pandas as pd

# Define the dataset
data = {
    "Milk":    [1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1],
    "Egg":     [1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
    "Bread":   [1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1],
    "Butter":  [1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0],
    "Ketchup": [0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1],
    "Cookies": [0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1],
}

# Create a DataFrame
df = pd.DataFrame(data)
print("Transaction Dataset:")
df
```

```
Transaction Dataset:
```

Out[47]:

	Milk	Egg	Bread	Butter	Ketchup	Cookies
0	1	1	1	1	0	0
1	1	1	0	1	1	0
2	0	0	1	1	1	0
3	1	0	1	1	0	0
4	0	0	1	1	0	1
5	1	0	1	1	0	1
6	1	0	0	0	0	1
7	1	0	1	1	0	0
8	0	1	1	1	0	1
9	1	0	1	1	0	0
10	1	0	1	1	0	0
11	1	0	1	0	1	1

c. Apply the Apriori Algorithm

We will use the `mlxtend` library to find frequent itemsets and generate association rules.

In [48]:

```
from mlxtend.frequent_patterns import apriori, association_rules
```

```
# Step 1: Generate frequent itemsets with Apriori
frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)

# Display frequent itemsets
print("\nFrequent Itemsets:")
print(frequent_itemsets)
```

```
Frequent Itemsets:
    support               itemsets
0  0.750000                 (Milk)
1  0.833333                (Bread)
2  0.833333               (Butter)
3  0.583333          (Bread, Milk)
4  0.583333         (Butter, Milk)
5  0.750000        (Butter, Bread)
6  0.500000  (Butter, Bread, Milk)
```

```
c:\Users\me\myenv\Lib\site-packages\mlxtend\frequent_patterns\fpcommon.py:161: D
eprecationWarning: DataFrames with non-bool types result in worse computationalp
erformance and their support might be discontinued in the future.Please use a Da
taFrame with bool type
  warnings.warn(
```

d. Generate Association Rules

We generate rules based on **confidence** and calculate **lift**.

In [49]:

```
# Step 2: Generate association rules
rules = association_rules(frequent_itemsets, num_itemsets=len(df), metric="con

# Display the rules
print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

```
Association Rules:
      antecedents consequents   support  confidence      lift
0         (Bread)      (Milk)  0.583333    0.700000  0.933333
1          (Milk)     (Bread)  0.583333    0.777778  0.933333
2        (Butter)      (Milk)  0.583333    0.700000  0.933333
3          (Milk)    (Butter)  0.583333    0.777778  0.933333
4        (Butter)     (Bread)  0.750000    0.900000  1.080000
5         (Bread)    (Butter)  0.750000    0.900000  1.080000
6  (Butter, Milk)     (Bread)  0.500000    0.857143  1.028571
7   (Bread, Milk)    (Butter)  0.500000    0.857143  1.028571
```

e. Filter and Interpret Rules

You can filter rules for high lift or specific itemsets.

In [50]:

```
# Filter rules with Lift > 1
filtered_rules = rules[rules['lift'] > 1]
print("\nFiltered Rules with Lift > 1:")
print(filtered_rules[['antecedents', 'consequents', 'support', 'confidence', '
```

```
Filtered Rules with Lift > 1:
```

```
   antecedents consequents  support  confidence      lift
4       (Butter)     (Bread)     0.75    0.900000  1.080000
5        (Bread)    (Butter)     0.75    0.900000  1.080000
6 (Butter, Milk)     (Bread)     0.50    0.857143  1.028571
7  (Bread, Milk)    (Butter)     0.50    0.857143  1.028571
```

Interpretation

1. $$\text{Milk, Bread} \rightarrow \text{Butter}$$

   - Support = 58%, Confidence = 87.5%, Lift = 1.05.
   - Suggest bundling these items for promotions.

2. $$\text{Milk, Butter} \rightarrow \text{Bread}$$

   - Similar metrics as above, showing strong relationships.